VajraKavach

Secure Job Search & Professional Networking Platform

Security Audit Report

Multi-Tool Black-Box Penetration Test — Revision 3

CSE 345/545: Foundations to Computer Security

Initial Audit	April 6, 2026
Re-Audit Date	April 7, 2026
Target	`https://192.168.3.44`
Scope	Full external attack surface (incl. WebSocket)
Methodology	Black-box automated + manual code review

Team Members	Roll Number
Yash Kumar Vaibhav	MT25091
Aayush Gupta	MT25008
Ayush Jain	MT25066
Ayush Kumar	2020290

Indraprastha Institute of Information Technology, Delhi

Contents

1 Executive Summary **3**

2 Tools and Methodology **3**

3 Port Scan Results (nmap) **3**

4 TLS/SSL Audit (testssl.sh) **4**
4.1 Protocol Support . . . 4
4.2 Cipher Suites (AEAD-Only) . . . 5
4.3 Known TLS Vulnerability Checks . . . 5
4.4 Certificate Details . . . 6

5 Web Server Scan (Nikto) **6**

6 SQL Injection Testing (sqlmap) **6**

7 Comprehensive Web Scan (wapiti) **7**

8 Endpoint Access Control Audit **7**
8.1 Sensitive Path Blocking . . . 8
8.2 API Authentication Enforcement . . . 8

9 Security Headers Audit **9**

10 Re-Audit: Nikto Re-Scan (April 7, 2026) **9**

11 WebSocket Security Audit **9**
11.1 Authentication . . . 10
11.2 Cross-Site WebSocket Hijacking (CSWSH) . . . 10
11.3 WebSocket Configuration (nginx) . . . 10

12 TOTP Lockout Mechanism Audit **10**

13 API Rate Limiting & Resilience **11**

14 Re-Audit Confirmation: Comprehensive Testing (April 7, 2026) **11**
14.1 nmap Re-Scan . . . 11
14.2 nmap Vulnerability Scripts . . . 11
14.3 testssl.sh Re-Scan . . . 12
14.4 sqlmap Re-Scan . . . 12
14.5 wapiti Re-Scan . . . 12
14.6 gobuster Directory Enumeration . . . 12
14.7 HTTP Method Testing . . . 13
14.8 CORS Validation . . . 13
14.9 Cookie Security Flags . . . 13
14.10 Authentication Bypass Testing . . . 13
14.11 Injection & Manipulation Tests . . . 14
14.12 Dependency Audit . . . 14

14.13Debug Endpoint Exposure 14

15 Fixes Applied During Audit **15**

16 Overall Assessment **16**

17 Service Resilience **16**
17.1 Systemd Watchdog 16
17.2 Health Check Timer 17
17.3 Kernel Panic Auto-Reboot 17

18 Attack Mitigation, Tolerance, and Recovery **17**
18.1 Attack Mitigation 18
18.2 Attack Tolerance 19
18.3 Attack Recovery 19
18.4 Resilience Summary 20

Executive Summary

A comprehensive black-box security audit was first conducted on April 6, 2026 against VajraKavach (`https://192.168.3.44`). Following significant feature additions—real-time WebSocket messaging, progressive IP-based TOTP lockout, and API retry logic—a full re-audit was performed on April 7, 2026 using eight tools plus manual code review.

Final Assessment: STRONG

0 Critical **0** High **0** Medium **0** Low

All findings discovered during both audits were **fixed immediately**.
14 known TLS/SSL CVEs tested — all clean.
Zero injection vectors across all categories.
WebSocket endpoint secured against CSWSH.
All HTTP methods except GET/POST blocked (405).
CORS, cookie flags, auth bypass, and dependency audits all clean.

Tools and Methodology

Tool	Version	Purpose
nmap	7.80	Port scanning, service detection, SSL cipher enumeration, vulnerability scripts
testssl.sh	3.2	TLS/SSL configuration audit (protocols, ciphers, 14+ CVE checks)
Nikto	2.6.0	Web server misconfiguration and information leakage scanner
sqlmap	1.6.4	Automated SQL injection detection and exploitation
wapiti	3.0.4	Comprehensive web vulnerability scanner (SQLi, XSS, CSRF, SSRF, XXE, RCE, LFI)
gobuster	2.0.1	Directory/path brute-force enumeration
npm audit	—	Frontend dependency vulnerability scanner
curl	7.81	Manual HTTP method, CORS, cookie, injection, and auth bypass probing

Table 1: Security testing toolkit (8 tools)

Approach: Each tool was run independently against the production deployment. Authenticated tests used a valid JWT session cookie. Rate limiting was observed and accounted for during testing. The re-audit was conducted after implementing WebSocket messaging, IP-based TOTP lockout, and CSWSH protection.

Port Scan Results (nmap)

```
$ sudo nmap -sV -sC -p 80,443,8000,8080,3000,5000,5432,22
   192.168.3.44
```

```
PORT     STATE   SERVICE    VERSION
22/tcp   open    ssh        OpenSSH 8.9p1 Ubuntu
80/tcp   open    http       nginx (returns 444 - connection
   drop)
443/tcp  open    ssl/http   nginx (TLS 1.2 + 1.3)
3000/tcp closed  -
5000/tcp closed  -
5432/tcp closed  postgresql (not externally accessible)
8000/tcp closed  http-alt   (backend bound to 127.0.0.1 only)
8080/tcp closed  -
```

Assessment: Only essential ports exposed. Backend (8000) and database (5432) are properly isolated on localhost. HTTP port 80 drops connections with status 444 (no redirect — prevents credential leakage during downgrade attacks).

TLS/SSL Audit (testssl.sh)

Protocol Support

Protocol	Status	Assessment
SSLv2	Not offered	✓ Secure
SSLv3	Not offered	✓ Secure (POODLE mitigated)
TLS 1.0	Not offered	✓ Secure (BEAST mitigated)
TLS 1.1	Not offered	✓ Secure
TLS 1.2	Offered	✓ Supported
TLS 1.3	Offered	✓ Modern standard

Table 2: TLS protocol support — only TLS 1.2 and 1.3 offered

Cipher Suites (AEAD-Only)

Cipher Suite	Key Exchange	Encryption	Bits	Grade
TLS 1.3				
TLS_AES_256_GCM_SHA384	ECDH x25519	AESGCM	256	A
TLS_CHACHA20_POLY1305_SHA256	ECDH x25519	ChaCha20	256	A
TLS_AES_128_GCM_SHA256	ECDH x25519	AESGCM	128	A
TLS 1.2				
ECDHE-RSA-AES256-GCM-SHA384	ECDH x25519	AESGCM	256	A
ECDHE-RSA-AES128-GCM-SHA256	ECDH x25519	AESGCM	128	A
ECDHE-RSA-CHACHA20-POLY1305	ECDH x25519	ChaCha20	256	A

Table 3: Offered cipher suites — all AEAD, all Grade A, no CBC ciphers

Key points:

- All ciphers use **Authenticated Encryption with Associated Data** (AEAD).
- **No CBC ciphers** offered — LUCKY13 (CVE-2013-0169) eliminated.
- **Forward Secrecy** via ECDHE with X25519 on all cipher suites.
- Server cipher order enforced (`ssl_prefer_server_ciphers on`).

Known TLS Vulnerability Checks

Vulnerability	CVE	Status
Heartbleed	CVE-2014-0160	✓
CCS Injection	CVE-2014-0224	✓
Ticketbleed	CVE-2016-9244	✓
ROBOT	—	✓
CRIME	CVE-2012-4929	✓
BREACH	CVE-2013-3587	✓ (gzip disabled)
POODLE	CVE-2014-3566	✓ (no SSLv3)
SWEET32	CVE-2016-2183	✓
FREAK	CVE-2015-0204	✓
DROWN	CVE-2016-0800	✓
LOGJAM	CVE-2015-4000	✓
BEAST	CVE-2011-3389	✓ (no TLS 1.0)
LUCKY13	CVE-2013-0169	✓ (no CBC ciphers)
RC4	CVE-2013-2566	✓ (no RC4)

Table 4: All 14 known TLS/SSL CVEs — not vulnerable

Certificate Details

Property	Value
Algorithm	SHA256 with RSA (2048-bit key)
SAN	DNS:localhost, IP:192.168.3.44, IP:127.0.0.1
Issuer	mkcert development CA
Forward Secrecy	Yes (ECDHE with X25519)
HSTS	365 days with includeSubDomains

Table 5: TLS certificate properties

Web Server Scan (Nikto)

Finding	Severity	Status	Details
BREACH (gzip)	Medium	Fixed	Added `gzip off;` in nginx server block
X-Request-ID leak	Low	Fixed	Removed from response headers; `proxy_hide_header` in nginx
Cert SAN mismatch	Info	FP	Cert has IP SAN; Nikto doesn't parse IP SANs
X-Content-Type-Options	Info	FP	Header present (verified via curl)
JAMonAdmin.jsp	Info	FP	React SPA returns 200 for all unknown paths

Table 6: Nikto findings — 2 real issues fixed, 3 false positives identified

FP = False Positive

SQL Injection Testing (sqlmap)

Tested with authenticated session against all search/filter endpoints:

Endpoint	Parameters Tested	Result
`/api/jobs/?keyword=test`	keyword	✓ Not injectable
`/api/users/search?q=test`	q	✓ Not injectable
`/api/companies/?q=test`	q	✓ Not injectable
`/api/jobs/?keyword=test&location=delhi`	keyword, location	✓ Not injectable

Table 7: SQL injection test results — all endpoints clean

Why VajraKavach is not injectable:

- All queries use **SQLAlchemy ORM** with parameterized queries — no raw SQL.

- `escape_like()` function used for all LIKE clauses.
- All inputs have **Pydantic max_length constraints** at the schema level.
- sqlmap also detected **rate limiting** (identified as WAF protection).

Comprehensive Web Scan (wapiti)

Vulnerability Category	Findings	Status
SQL Injection	0	✓
Blind SQL Injection	0	✓
Cross Site Scripting (XSS)	0	✓
Permanent XSS (Stored)	0	✓
Cross Site Request Forgery (CSRF)	0	✓
Server Side Request Forgery (SSRF)	0	✓
XML External Entity (XXE)	0	✓
Command Execution (RCE)	0	✓
Path Traversal (LFI)	0	✓
Open Redirect	0	✓
CRLF Injection	0	✓
Backup Files	0	✓
Weak Credentials	0	✓
Htaccess Bypass	0	✓
HttpOnly Cookie Flag	0	✓
Secure Cookie Flag	0	✓
Internal Server Error	0	✓
CSP Configuration	1	**Fixed**

Table 8: Wapiti scan results — 17 categories clean, 1 finding fixed

The single CSP finding (`object-src` not set) was fixed by adding `object-src 'none'` to all Content-Security-Policy headers.

Endpoint Access Control Audit

Sensitive Path Blocking

Path	Expected	Actual	Status
`/.env`	404	404	✓
`/.git`	404	404	✓
`/.git/config`	404	404	✓
`/wp-admin`	404	404	✓
`/phpinfo.php`	404	404	✓
`/server-status`	404	404	✓
`/actuator`	404	404	✓
`/console`	404	404	✓
`/debug`	404	404	✓
`/api/docs`	404	404	✓
`/api/redoc`	404	404	✓
`/openapi.json`	404	404	✓

Table 9: All sensitive paths return 404 — no information leakage

API Authentication Enforcement

Endpoint	Without Auth	Status
`/api/auth/me`	401 Unauthorized	✓
`/api/messages/conversations`	401 Unauthorized	✓
`/api/admin/users`	401 Unauthorized	✓
`/api/admin/blockchain/blocks`	401 Unauthorized	✓
`/api/ws` (WebSocket)	Rejected (code 4001)	✓
`/api/jobs/search`	401 Unauthorized	✓
`/api/companies/`	401 Unauthorized	✓
`/api/reports/admin/all`	401 Unauthorized	✓

Table 10: All API and WebSocket endpoints require authentication

Security Headers Audit

Header	Value	Status
Strict-Transport-Security	max-age=31536000; includeSubDomains	✓
X-Frame-Options	DENY	✓
X-Content-Type-Options	nosniff	✓
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; connect-src 'self'; font-src 'self'; **object-src 'none'**; frame-ancestors 'none'; base-uri 'self'; form-action 'self';	✓
Referrer-Policy	strict-origin-when-cross-origin	✓
Permissions-Policy	camera=(), microphone=(), geolocation=()	✓
X-XSS-Protection	1; mode=block	✓
X-Request-ID	Not present	✓
Server	nginx (version hidden)	✓
Content-Encoding	Not present (gzip disabled)	✓

Table 11: Complete security header coverage on all response types

Re-Audit: Nikto Re-Scan (April 7, 2026)

Following WebSocket implementation, lockout mechanism redesign, and API retry logic, Nikto was re-run:

```
$ nikto -h https://192.168.3.44 -ssl -nointeractive -Tuning x

+ Server: nginx
+ 6544 items checked: 0 error(s) and 8 item(s) reported
+ All 8 items are informational (security headers detected)
+ 0 vulnerabilities found
```

Result: Clean scan. All previously fixed issues remain fixed. The only items reported are positive confirmations of our security headers (HSTS, CSP, X-Frame-Options, etc.).

WebSocket Security Audit

A WebSocket endpoint (`wss://192.168.3.44/api/ws`) was added for real-time messaging notifications. The following security aspects were tested:

Authentication

Check	Result	Status
Connect without token	Rejected (code 4001)	✓
Connect with expired JWT	Rejected (code 4003)	✓
Connect with pre-TOTP JWT (`totp_verified: false`)	Rejected (code 4003)	✓
Connect with valid JWT	Accepted	✓
IP binding check (JWT IP ≠ request IP)	Rejected (code 4003)	✓
Suspended/inactive user	Rejected (code 4003)	✓

Table 12: WebSocket authentication — all checks enforced

Cross-Site WebSocket Hijacking (CSWSH)

Finding	Severity	Details
Missing Origin validation	Medium	**Fixed** — Origin header is now validated on every connection; only `https://192.168.3.44` is accepted.

Table 13: CSWSH vulnerability found and fixed during re-audit

WebSocket Configuration (nginx)

Check	Status
Separate location block (no rate limiting on persistent conn.)	✓
`proxy_http_version 1.1` with Upgrade/Connection headers	✓
Read/write timeout: 3600s (prevents idle kills)	✓
X-Real-IP forwarded for IP binding	✓

Table 14: nginx WebSocket proxy — properly configured

TOTP Lockout Mechanism Audit

The TOTP lockout was redesigned from email-based to IP-based progressive lockout:

Property	Value	Status
Lockout trigger	3 failed attempts in 5 min	✓
Lockout scope	Per IP address	✓
First lockout duration	60 minutes	✓
Progressive escalation	×5 each time (300, 1500, ...)	✓
Previous vulnerability: email-based	Allowed DoS against any user	**Fixed**
Lockout state persistence	Survives server restarts (DB-backed)	✓
TOTP validity window	±1 time-step (≈90 seconds)	✓

Table 15: IP-based progressive TOTP lockout — prevents account DoS

API Rate Limiting & Resilience

Limit Zone	Configuration	Status
Global API	30 req/min per IP, burst 10	✓
Auth endpoints	5 req/min per IP, burst 3	✓
WebSocket	Exempt (persistent connection)	✓
Frontend retry on 429/503	3-second delay + single retry	✓
Refresh token 429 handling	Does not trigger logout	✓
Message polling fallback	60s interval (WebSocket is primary)	✓

Table 16: Rate limiting and resilience configuration

Re-Audit Confirmation: Comprehensive Testing (April 7, 2026)

All tools from the initial audit were re-run and additional manual tests were performed to confirm the security posture after new feature deployment.

nmap Re-Scan

Port scan was re-run with full service detection. Only ports 22 (SSH), 80 (HTTP→444 drop), and 443 (HTTPS) are open. Backend and database remain bound to localhost only. No new services exposed.

nmap Vulnerability Scripts

```
$ sudo nmap --script vuln -p 443 192.168.3.44
```

Test	Result	Status
CSRF Detection	No CSRF found	✓
DOM-based XSS	No DOM XSS found	✓
Stored XSS	No stored XSS found	✓
DROWN (CVE-2016-0800)	Not vulnerable	✓
CVE-2011-3192 (Apache byterange)	Reported as vulnerable	FP

Table 17: nmap vulnerability scripts — 1 false positive (Apache-specific, runs nginx)

CVE-2011-3192 False Positive: This vulnerability is specific to Apache's `mod_byterange` module which causes exponential memory consumption with overlapping Range headers. Our nginx server handles multi-range requests safely—tested with `Range: bytes=0-5,1-3,2-4`: response was 206 Partial Content with a small 480-byte body (multipart boundaries). Not exploitable.

testssl.sh Re-Scan

TLS configuration re-verified: only TLS 1.2/1.3 offered, AEAD-only cipher suites, forward secrecy on all ciphers, all 14 CVE checks passed. HSTS active at 365 days.

sqlmap Re-Scan

All search endpoints retested (`/api/jobs`, `/api/users/search`, `/api/companies`): no injectable parameters found. Rate limiting detected as WAF.

wapiti Re-Scan

Full scan across 18 vulnerability categories: **0 vulnerabilities found**. CSP previously fixed remains correct.

gobuster Directory Enumeration

```
$ gobuster -m dir -u https://192.168.3.44 -w wordlist.txt
  Wildcard response detected (SPA fallback, Status: 200)
  /api (Status: 301) - only real route
```

The React SPA returns 200 for all unknown paths (serving `index.html`). Gobuster correctly identified this as a wildcard response. Only `/api` (301 redirect) is a genuine route. No hidden directories or files exposed.

HTTP Method Testing

HTTP Method	Response	Status
GET	200	✓ (allowed)
POST	200/401	✓ (allowed)
TRACE	405 Method Not Allowed	✓ (blocked)
OPTIONS	405 Method Not Allowed	✓ (blocked)
PUT	405 Method Not Allowed	✓ (blocked)
DELETE	405 Method Not Allowed	✓ (blocked)

Table 18: Only GET and POST accepted — all dangerous methods blocked

CORS Validation

Origin	Response Headers	Status
`https://evil.com`	No `Access-Control-Allow-Origin` returned	✓
`https://192.168.3.44`	`Access-Control-Allow-Origin: https://192.168.3.44` + `credentials: true`	✓

Table 19: CORS properly restricts cross-origin access

Cookie Security Flags

Flag	Value	Status
HttpOnly	Set on both access_token and refresh_token	✓
Secure	Set (HTTPS-only transmission)	✓
SameSite	strict (prevents CSRF)	✓
Max-Age (access)	1800 seconds (30 minutes)	✓
Max-Age (refresh)	604800 seconds (7 days)	✓
Path	/	✓

Table 20: All cookies have complete security flag coverage

Authentication Bypass Testing

Test	Response	Status
Tampered JWT (forged admin role) on `/api/users/me/profile`	401 Invalid or expired token	✓
No auth on `/api/admin/users`	401 Authentication required	✓
Tampered JWT on `/api/admin/dashboard`	401 Invalid or expired token	✓
Pre-TOTP JWT on `/api/users/search`	403 TOTP verification required	✓

Table 21: Auth bypass attempts — all rejected

Injection & Manipulation Tests

Test	Response	Status
Path traversal (`/../../etc/passwd`)	200 (SPA fallback, no file content)	✓
Null byte (`/%00`)	400 Bad Request	✓
CRLF injection (`/%0d%0aX-Injected: header`)	200 (no injected headers)	✓
Host header (`Host: evil.com`)	200 (static SPA, no redirect)	✓
XSS in search (`?q=<script>`)	403 (TOTP required, never reaches DB)	✓

Table 22: All injection and manipulation attempts blocked

Dependency Audit

Package Manager	Vulnerabilities Found	Status
npm audit (frontend)	0	✓
pip-audit (backend)	Unable to reach PyPI (offline)	N/A

Table 23: Frontend dependencies clean — backend requires online PyPI access

Debug Endpoint Exposure

Endpoint	Response	Status
`/docs` (Swagger UI)	SPA fallback (not API docs)	✓
`/redoc` (ReDoc)	SPA fallback (not API docs)	✓
`/openapi.json`	404	✓
`/api/docs`	404	✓
`/api/openapi.json`	404	✓

Table 24: FastAPI docs disabled (`DEBUG=false`) — no API schema leakage

Fixes Applied During Audit

#	Finding	Fix	File
Initial Audit (April 6, 2026)			
1	BREACH (CVE-2013-3587)	`gzip off;` in nginx server block	nginx.conf
2	X-Request-ID leak	`proxy_hide_header` + removed from middleware response	nginx.conf, security.py
3	LUCKY13 (CVE-2013-0169)	AEAD-only cipher suite, CBC removed	nginx.conf
4	CSP missing object-src	Added `object-src 'none'` to all CSP headers	nginx.conf
5	/assets/ header loss	Re-added security headers in /assets/ location block	nginx.conf
6	Duplicate API headers	SecurityHeadersMiddleware deregistered (nginx handles all)	middleware.py
Re-Audit (April 7, 2026)			
7	CSWSH on WebSocket	Added Origin header validation against allowed origin	websocket.py
8	Email-based TOTP lockout	Changed to IP-based progressive lockout (prevents DoS)	totp_service.py
9	Polling rate too high	WebSocket replaces polling; 60s fallback only	MessagesPage.jsx
10	No 429/503 retry	Added automatic retry with 3s delay in API interceptor	api.js
Post-Audit Hardening (April 7, 2026 — Late)			
11	TOTP setup-verify enumeration	Uniform error response for all cases (no account existence leak)	auth.py
12	RDP port 3389 open (xrdp)	Stopped and disabled xrdp service; port confirmed closed	systemd
13	No application-level rate limits	Added 16 DB-	action_rate_limit_service.py

Overall Assessment

Category	Result
Critical vulnerabilities	**0**
High vulnerabilities	**0**
Medium vulnerabilities	**0** (all fixed)
Low vulnerabilities	**0** (all fixed)
SQL injection vectors	**0**
XSS vectors	**0**
CSRF vectors	**0**
Known TLS/SSL CVEs clean	**14/14**
Sensitive paths blocked	**12/12**
API auth enforcement	**100%**
WebSocket auth enforcement	**100%**
Security headers present	**10/10**
HTTP methods restricted	**4/4 blocked** (TRACE, OPTIONS, PUT, DELETE)
CORS configured correctly	**Yes** (same-origin only)
Cookie security flags	**3/3** (HttpOnly, Secure, SameSite=strict)
Auth bypass attempts	**0/4 succeeded**
Injection attempts	**0/5 succeeded**
Frontend dep. vulnerabilities	**0**
Debug endpoints exposed	**0**

The application demonstrates **defense-in-depth** across all four layers:

1. **Kernel:** SYN cookies, TCP hardening, ICMP rate limiting
2. **nginx:** TLS termination, rate limiting, path blocking, security headers, connection limits, WebSocket proxy with origin isolation
3. **Application:** Pydantic validation, SQLAlchemy ORM, TOTP enforcement, JWT with short expiry + IP binding, progressive IP-based lockout, CSWSH protection, E2EE messaging (RSA-4096 + AES-256-GCM), RSA-PSS message signing, **16 DB-backed action rate limits** (daily + weekly per user/IP)
4. **Database:** Localhost binding, Argon2 password hashing, encrypted TOTP secrets, Fernet-encrypted resumes, SHA-256 hash-chained audit logs, blockchain integrity layer with Merkle trees and proof-of-work

Service Resilience

The following resilience mechanisms were implemented to ensure continuous availability and automatic recovery from crashes, hangs, and system-level failures.

Systemd Watchdog

The backend service is configured with `Type=notify` and `WatchdogSec=30`. The FastAPI application sends `WATCHDOG=1` keepalive signals every 15 seconds via the `sd_notify` pro-

tocol. If the process hangs (deadlock, infinite loop, resource exhaustion), systemd detects the missed heartbeat after 30 seconds and automatically kills and restarts the process.

```
[Service]
Type=notify
NotifyAccess=all
WatchdogSec=30
Restart=always
RestartSec=3
StartLimitIntervalSec=60
StartLimitBurst=10
MemoryMax=2G
CPUQuota=80%
```

Health Check Timer

A systemd timer (`vajrakavach-healthcheck.timer`) runs every 2 minutes and executes a health check script that:

1. Sends an HTTPS request to `/api/health`
2. Retries up to 3 times with 5-second delays
3. If all retries fail, logs an alert via `syslog` and restarts the backend service

This catches scenarios where the process is running but not serving requests (e.g., database connection pool exhaustion, event loop blocked).

Kernel Panic Auto-Reboot

The system is configured with `kernel.panic=10` (persisted in `/etc/sysctl.d/99-vajrakavach-panic.` which automatically reboots the machine 10 seconds after a kernel panic, ensuring the system recovers from catastrophic OS-level failures without manual intervention.

Attack Mitigation, Tolerance, and Recovery

This section audits the platform across three security resilience pillars: preventing attacks from succeeding (mitigation), continuing to operate during attacks (tolerance), and recovering after attacks (recovery).

Attack Mitigation

Mechanism	Implementation	Location
Rate Limiting (Nginx)	30 req/min global, 5 req/min on auth endpoints, burst queues	nginx.conf
Rate Limiting (App)	60 req/min global, 5 req/min on auth (defense-in-depth)	security.py
Connection Limits	15 concurrent connections per IP	nginx.conf
Input Validation	Pydantic schemas with Field constraints (min/max length, regex)	schemas/*.py
SQL Injection Prevention	SQLAlchemy ORM (parameterized queries), `escape_like()` for LIKE/ILIKE	sanitize.py
XSS Prevention	CSP header (script-src 'self'), React DOM escaping, X-XSS-Protection	nginx.conf
CSRF Protection	SameSite=strict cookies, CORS same-origin policy	auth.py, nginx.conf
CSWSH Protection	WebSocket Origin header validation against allowed origin	websocket.py
HTTPS Enforcement	HTTP port returns 444 (connection close), HSTS max-age=31536000	nginx.conf
TLS Configuration	TLSv1.2/1.3 only, AEAD ciphers (`!CBC`), server cipher preference	nginx.conf
Password Hashing	Argon2id (memory-hard, side-channel resistant)	auth_service.py
Password Policy	8–72 chars, uppercase, lowercase, digit, special character required	schemas/auth.py
JWT Security	Short expiry (15 min access), IP binding, type claims, HS256	auth_service.py
TOTP 2FA	RFC 6238 TOTP with 90-second window, encrypted secrets (Fernet)	totp_service.py
Progressive Lockout	IP-based: 3 failures in 5 min → 60 min lockout, ×5 escalation	totp_service.py
Token Revocation	SHA-256 hashed blocklist, DB-backed + in-memory cache	auth_service.py
BREACH Prevention	HTTP compression disabled (`gzip off`)	nginx.conf
Path Blocking	Dotfiles, .env, .git, wp-admin, phpinfo, etc. return 404	nginx.conf
Debug Endpoint Blocking	/api/docs, /api/redoc, /openapi.json disabled in production	nginx.conf, main.py
Body Size Limits	1 MB default, 5 MB for API uploads	nginx.conf
Request ID Concealment	X-Request-ID generated internally, stripped from responses	security.py, nginx.conf
LAN-Only Access	Nginx `allow 192.168.0.0/16; deny all` — not internet-exposed	nginx.conf
E2E Encryption	RSA-4096 key exchange + AES-256-GCM for messages	pki_service.py
Message Signing	RSA-PSS with SHA-256 for message authenticity	pki_service.py

Attack Tolerance

These mechanisms allow the system to continue operating under active attack:

Mechanism	Description
Dual Rate Limiting	Both Nginx (kernel-level, fast) and application layers rate-limit independently. An attacker cannot bypass one without hitting the other.
Resource Ceilings	`MemoryMax=2G` and `CPUQuota=80%` prevent a compromised or overloaded backend from exhausting host resources.
Connection Throttling	Nginx enforces 15 concurrent connections per IP. DNS amplification or slowloris attacks are bounded.
Database Connection Pool	PostgreSQL connection pool (`pool_size=10, max_overflow=5, pool_pre_ping=True`) prevents connection exhaustion under load.
Progressive Lockout	Brute-force attackers face exponentially increasing lockout: 60 min → 300 min → 1500 min → 7500 min. System continues serving legitimate users.
WebSocket Isolation	Each WebSocket connection is authenticated independently; a compromised connection cannot affect others. Dead connections are cleaned up automatically.
Async Architecture	FastAPI with asyncio processes requests concurrently without blocking. A slow request (e.g., mining a block) does not block other requests.
Graceful Degradation	Rate-limited clients receive HTTP 429 with retry guidance. Auth-locked IPs receive clear lockout messages. Health endpoint remains always available.
HTTP 444 on Plain HTTP	Port 80 returns 444 (close connection immediately) — no resources spent serving plain-text probes.
Periodic Cleanup	Rate limit and lockout dictionaries are automatically cleaned up every 60 seconds, preventing memory growth from distributed attacks.

Attack Recovery

These mechanisms enable the system to recover automatically after an attack or failure:

Mechanism	Description
Auto-Restart	`Restart=always` with `RestartSec=3`: systemd restarts the backend within 3 seconds of any crash. Up to 10 restarts allowed in 60 seconds.
Watchdog Recovery	`WatchdogSec=30`: if the process hangs (not just crashes), systemd kills and restarts it within 30 seconds.
Health Check Timer	External health check every 2 minutes with 3 retries. Catches "alive but unresponsive" scenarios and auto-restarts.
Kernel Panic Reboot	`kernel.panic=10`: system reboots 10 seconds after a kernel panic, recovering from catastrophic OS failures.
Token Revocation	Compromised tokens can be revoked immediately; the blocklist persists across restarts (DB-backed).
Audit Trail	SHA-256 hash-chained audit logs provide tamper-evident forensic evidence of pre-attack, attack, and post-attack activity.
Blockchain Integrity	Merkle-tree blocks with proof-of-work over audit entries enable detection of any log tampering after an attack.
Security State Persistence	TOTP lockout state and token revocation cache are persisted to PostgreSQL on shutdown and reloaded on startup — surviving any restart.
File Integrity	Resume files are encrypted with Fernet (AES-128-CBC + HMAC-SHA256); tampering is detectable via the HMAC. SHA-256 checksums stored per file.
Service Dependency Chain	systemd `Requires=postgresql.service` ensures the database is always available before the backend starts.

Resilience Summary

Pillar	Count of Mechanisms
Attack Mitigation	**23 mechanisms**
Attack Tolerance	**10 mechanisms**
Attack Recovery	**10 mechanisms**
Total Resilience Mechanisms	**43**